

AI ACADEMY, NATIONAL AI CENTER
Machine Learning – Spring 2026

Final Project Brief

Ensemble Methods: Boosting vs. Bagging

Released:	June 30, 2026	Due:	July 16, 2026 at 23:59 (UTC+4)
Team size:	3 students (2 or 4 with approval)	Workload:	~35–40 person-hours over 12 days
Version:	v3.0	Submission:	GitHub repo + Moodle + oral defense

At a glance

What this project is. You will implement, from scratch, three machine learning algorithms: a **Decision Tree**, an **AdaBoost classifier** (which uses a depth-1 Decision Tree — a decision stump — as its weak learner), and a **Random Forest** (which uses bagging over full Decision Trees). You will then run a controlled empirical study comparing the two ensemble philosophies — boosting vs. bagging — using real-world datasets, and support your analysis with **unsupervised learning techniques** (PCA, K-Means, DBSCAN) to explore and visualize the data before and after classification. **What it is NOT.** You may not use `sklearn`’s `DecisionTreeClassifier`, `AdaBoostClassifier`, or `RandomForestClassifier` as your primary implementations. You may use them *only* as a sanity-check baseline to compare against your own output. **Four deliverables, one deadline.** On **July 16, 2026 at 23:59 (UTC+4)** you submit one archive to Moodle containing:

1. Source **code** (GitHub tag `v1.0-final`)
2. Compiled **paper** (`report/report.pdf`), IEEE style
3. Professional **slide deck** (`presentation/presentation.pdf`)
4. A scheduled **oral defense** (presentation with slides)

Contents

1	Motivation and Background Reading	2
1.1	Required Reading	2
2	What You Will Implement	3
2.1	Module 1 — Decision Tree Classifier	3
2.1.1	Interface contract	3
2.1.2	Detailed Implementation Steps	4
2.1.3	Testing and Sanity Checks	5
2.2	Module 2 — AdaBoost with Decision Stumps	5
2.2.1	Interface contract	5
2.2.2	Mathematical Algorithm	5
2.2.3	Implementation Details	6
2.3	Module 3 — Random Forest	6
2.3.1	Interface contract	6
2.3.2	Algorithm Steps	7
2.4	Module 4 — Unsupervised Analysis Pipeline	7

2.4.1	PCA	7
2.4.2	K-Means	7
2.4.3	DBSCAN	8
3	Experimental Study Design	8
3.1	Datasets	8
3.2	Preprocessing	9
3.3	Experiments to run	9
4	Project Format	10
4.1	Teams	10
4.2	Repository structure	10
5	Deliverables	11
6	Contribution Report	11
7	Grading Rubric	12
7.1	Automatic deductions	12
8	Advanced Bonuses (Optional)	13
9	What You May NOT Do	13
10	Academic Integrity	13
11	Self-Assessment Checklist	13

1. Motivation and Background Reading

The central question this project asks you to answer empirically is:

“Under what conditions does boosting outperform bagging, and vice versa, and why?”

To answer it rigorously you must first understand what each approach does at a mathematical level. The references below are **required reading** before you write a single line of code. Rather than original papers alone, we point you to the relevant chapters of *The Elements of Statistical Learning* (ESL, Hastie, Tibshirani & Friedman, 2nd ed., freely available at <https://hastie.su.domains/ElemStatLearn/>) as the primary source, supplemented by the original papers where the ESL treatment is thin.

1.1 Required Reading

1. Decision Trees (CART)

ESL Chapter 9.2 (“Tree-Based Methods”).

Original reference (supplementary): Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.

<https://link.springer.com/article/10.1007/BF00116251>

2. AdaBoost and Boosting

ESL Chapter 10 (“Boosting and Additive Trees”), Sections 10.1–10.6.

Original reference (supplementary): Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.

<https://www.sciencedirect.com/science/article/pii/S002200009791504X>

3. Bagging and Random Forests

ESL Chapter 8.7 (“Bagging”) and Chapter 15 (“Random Forests”).

Original references (supplementary):

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.

<https://link.springer.com/article/10.1007/BF00058655>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://link.springer.com/article/10.1023/A:1010933404324>

4. Bias–Variance Tradeoff

ESL Chapter 7.3 (“The Bias–Variance Decomposition”). <https://www.sas.upenn.edu/~fdiebold/NoHesitations/BookAdvanced.pdf>

5. PCA

ESL Chapter 14.5.1 (“Principal Components”).

Original reference (supplementary): Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572. <https://www.sas.upenn.edu/~fdiebold/NoHesitations/BookAdvanced.pdf>

6. K-Means Clustering

ESL Chapter 14.3 (“Cluster Analysis”), Section 14.3.6 (“K-Means Clustering”).

Original reference (supplementary): Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

<https://ieeexplore.ieee.org/document/1056489>

7. DBSCAN

ESL does not cover DBSCAN in depth; read the original paper directly:

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd ACM SIGKDD*, 226–231.

<https://dl.acm.org/doi/10.5555/3001460.3001507>

Read the chapters

The oral defense will include questions drawn directly from the material above. You will be expected to explain: the AdaBoost weight update rule, why bootstrapping reduces variance, the Gini impurity criterion, and the difference between density-based and centroid-based clustering. Questions will only cover topics addressed in the course presentations and the reading list above.

2. What You Will Implement

2.1 Module 1 — Decision Tree Classifier

Implement a **binary Decision Tree** for classification from scratch as a Python class. The tree must handle continuous features only (no categorical splits with encoding tricks). The algorithm follows the CART (Classification and Regression Trees) methodology.

2.1.1 Interface contract

```

1 class DecisionTree:
2     def __init__(
3         self,
4         max_depth: int = None,
5         min_samples_split: int = 2,
6         criterion: str = "gini",           # "gini" or "entropy"
7         max_features: int | str | None = None, # for Random Forest
8         random_state: int | None = None,

```

```

9      ) -> None: ...
10     def fit(self, X: np.ndarray, y: np.ndarray) -> "DecisionTree": ...
11     def predict(self, X: np.ndarray) -> np.ndarray: ...
12     def predict_proba(self, X: np.ndarray) -> np.ndarray: ...
13     # Introspection (useful for your analysis)
14     @property
15     def depth(self) -> int: ...
16     @property
17     def n_leaves(self) -> int: ...
18     def feature_importances(self) -> np.ndarray: ...

```

2.1.2 Detailed Implementation Steps

DT.1 Node structure. Each node stores:

- **feature_index** (int): which feature to split on
- **threshold** (float): split threshold
- **left, right** (Node): child nodes
- **value** (np.ndarray of shape [n_classes]): class distribution at leaf; for internal nodes, this is the prediction if the tree were truncated at this depth (useful for **predict_proba**).
- **samples** (int): number of training samples that reach this node.

DT.2 Splitting criterion.

- **Gini impurity:** $I_G(p) = 1 - \sum_{c=1}^C p_c^2$.
- **Information gain (entropy):** $I_E(p) = - \sum_{c=1}^C p_c \log_2(p_c + \epsilon)$.
- **Impurity reduction:** $\Delta I = I(\text{parent}) - \frac{N_{\text{left}}}{N} I(\text{left}) - \frac{N_{\text{right}}}{N} I(\text{right})$.
- Choose the split (feature j , threshold t) that maximizes ΔI .

DT.3 Finding the best split (exhaustive search).

- For each continuous feature j :
 - (a) Sort the unique values of $X[:,j]$.
 - (b) Consider thresholds halfway between consecutive distinct values: $t = (v_k + v_{k+1})/2$.
 - (c) For each threshold, compute the impurity reduction. To avoid scanning all samples each time, maintain sorted order and accumulate counts incrementally (efficient $O(N \log N)$ per feature).
 - (d) If **max_features** is set, randomly select a subset of features (without replacement) at each node, using **random_state** for reproducibility.
- Handle the degenerate case: if no split reduces impurity, or if all samples belong to one class, or **min_samples_split** is violated, the node becomes a leaf.

DT.4 Stopping criteria.

- **max_depth** reached.
- Number of samples at node $< \text{min_samples_split}$.
- All samples have identical class labels (pure node).
- All samples have identical feature vectors (no valid split).

DT.5 Prediction.

- **predict**: traverse tree, return the majority class (argmax of class counts in leaf value).
- **predict_proba**: return normalized class counts (empirical probabilities).

DT.6 Feature importances. Compute the normalized total impurity reduction attributed to each feature across all splits in the tree. (Sum of ΔI for each split, divided by total reduction.)

DT.7 Repr. `__repr__` for trees of depth ≤ 4 prints an indented text tree showing feature, threshold, gini/entropy value, samples, and class distribution at each node.

2.1.3 Testing and Sanity Checks

- Unit test on a tiny 2D dataset (e.g., XOR, or two Gaussians) with known split.
- Compare accuracy, depth, and feature importances against `sklearn.tree.DecisionTreeClassifier` (using same parameters, excluding random seed differences). Agreement must be within 2%.
- Test edge cases: dataset with single feature, all identical labels, `min_samples_split=1`, `max_depth=0` (stump).

2.2 Module 2 — AdaBoost with Decision Stumps

Implement **AdaBoost** (the discrete SAMME variant, suitable for binary classification) using depth-1 `DecisionTree` instances (decision stumps) as weak learners.

2.2.1 Interface contract

```

1 class DecisionStump(DecisionTree):
2     """Convenience subclass: max_depth=1, single binary split."""
3     def __init__(self, criterion: str = "gini", random_state: int | None = None) ->
4         None:
5         super().__init__(max_depth=1, criterion=criterion, random_state=
6             random_state)
7 class AdaBoostClassifier:
8     def __init__(
9         self,
10         n_estimators: int = 50,
11         learning_rate: float = 1.0,
12         criterion: str = "gini",
13         random_state: int | None = None,
14     ) -> None: ...
15     def fit(self, X: np.ndarray, y: np.ndarray) -> "AdaBoostClassifier": ...
16     def predict(self, X: np.ndarray) -> np.ndarray: ...
17     def predict_proba(self, X: np.ndarray) -> np.ndarray: ...
18     # For analysis
19     @property
20     def estimator_weights(self) -> np.ndarray: ...
21     @property
22     def estimator_errors(self) -> np.ndarray: ...
23     def staged_predict(self, X: np.ndarray) -> Iterator[np.ndarray]: ...

```

2.2.2 Mathematical Algorithm

Initialize sample weights $w_i^{(1)} = 1/N$ for $i = 1, \dots, N$. For $m = 1$ to M (`n_estimators`):

- Fit a stump $h_m(x)$ to the training data weighted by $w^{(m)}$. Because the stump must handle weighted samples, you need to modify your `DecisionTree` fit to accept an argument `sample_weight`. Weighted impurity is computed using weighted class counts: $p_c = \sum_{i: y_i=c} w_i / \sum_i w_i$.
- Compute weighted error: $\epsilon_m = \sum_{i=1}^N w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i] / \sum_i w_i^{(m)}$.

- (iii) If $\epsilon_m = 0$, set $\epsilon_m = 10^{-10}$ (clipping to avoid division by zero). If $\epsilon_m \geq 0.5$, stop the algorithm (or raise `ValueError` if early termination is not acceptable for analysis; you may optionally continue by resetting weights, but document the choice).
- (iv) Compute estimator weight: $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m} + \ln(K-1)$, where K is number of classes. For binary classification $K = 2$ this simplifies to $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m}$.
- (v) Update sample weights: $w_i^{(m+1)} = w_i^{(m)} \exp(\alpha_m \cdot \mathbf{1}[h_m(x_i) \neq y_i])$, then normalize.
- (vi) Store h_m and α_m .

Final prediction: $\hat{y} = \arg \max_k \sum_{m=1}^M \alpha_m \mathbf{1}[h_m(x) = k]$. For `predict_proba`, soft voting is often done via weighted average of probabilities, but for discrete SAMME you can compute class-wise sums of α_m for each class prediction and normalize (exponential softmax of weighted votes may also be used; document your approach).

2.2.3 Implementation Details

- `fit` must accept an optional `sample_weight` parameter in the stump. Modify your `DecisionTree` class to incorporate weights (default uniform). Weighted Gini: $I_G = 1 - \sum_c (\sum_{i:y_i=c} w_i / \sum_i w_i)^2$.
- The stump training at each iteration must respect `random_state` passed to AdaBoost. Seed a separate generator per round based on `random_state + m`.
- `staged_predict`: yields predictions after each boosting round. For example, list of arrays [prediction after round 1, after round 2, ...].
- Store `estimator_weights` (α_m array) and `estimator_errors` (ϵ_m array).

2.3 Module 3 — Random Forest

Implement **Random Forest** using bootstrap aggregation (bagging) over `DecisionTree` instances with feature sub-sampling.

2.3.1 Interface contract

```

1 class RandomForestClassifier:
2     def __init__(
3         self,
4         n_estimators: int = 100,
5         max_depth: int | None = None,
6         max_features: int | str = "sqrt",    # int, "sqrt", "log2", or None
7         min_samples_split: int = 2,
8         bootstrap: bool = True,
9         oob_score: bool = False,
10        n_jobs: int = 1,                    # parallelism with multiprocessing
11        random_state: int | None = None,
12    ) -> None: ...
13    def fit(self, X: np.ndarray, y: np.ndarray) -> "RandomForestClassifier": ...
14    def predict(self, X: np.ndarray) -> np.ndarray: ...
15    def predict_proba(self, X: np.ndarray) -> np.ndarray: ...
16    @property
17    def oob_score_(self) -> float: ...      # if oob_score=True
18    @property
19    def feature_importances_(self) -> np.ndarray: ...

```

2.3.2 Algorithm Steps

- RF.1 Bootstrap sampling.** For each tree $t = 1 \dots T$, draw a bootstrap sample $\mathcal{D}_t$ of size N from the original dataset $\mathcal{D}$ with replacement. Record the out-of-bag (OOB) indices: samples not included in $\mathcal{D}_t$.
- RF.2 Tree training.** Train a `DecisionTree` with parameters `max_depth`, `min_samples_split`, `max_features` (e.g., "sqrt" means $\lfloor \sqrt{p} \rfloor$ features randomly chosen at each split). Set `random_state` derived from the forest's seed and tree index to ensure reproducibility.
- RF.3 Parallelization.** When `n_jobs > 1`, use `multiprocessing.Pool` to parallelize the training loop over trees. Ensure random states are properly set for each worker to guarantee identical results across runs with the same `random_state`. Implement a sequential baseline for timing comparison.
- RF.4 OOB score.** If `oob_score=True`, compute OOB predictions: for each sample i , aggregate predictions only from trees for which i was OOB. Majority vote gives OOB class; OOB accuracy is fraction correct. Store as `oob_score_`.
- RF.5 Prediction.** `predict`: majority vote across trees. `predict_proba`: average of probability vectors from each tree.
- RF.6 Feature importances.** Average the feature importances of all trees.

2.4 Module 4 — Unsupervised Analysis Pipeline

Implement three algorithms from scratch. Use NumPy only (except for the ARI metric which may be imported from `sklearn.metrics`). This pipeline is mandatory and will be graded; its findings must be discussed in the report. The interfaces below show the expected methods, their arguments, return types, and a short description. **Do not copy-paste implementations; you must write the logic yourself.**

2.4.1 PCA

```

1 class PCA:
2     # n_components: int -- number of principal components to retain
3     # .components_ : np.ndarray (n_components, n_features) -- top eigenvectors
4     # .explained_variance_ratio_ : np.ndarray (n_components,) -- variance fraction
    per component
5     def __init__(self, n_components: int) -> None: ...
6     def fit(self, X: np.ndarray) -> "PCA": ...           # compute cov matrix,
    eigen-decomp
7     def transform(self, X: np.ndarray) -> np.ndarray: ... # project onto
    components
8     def fit_transform(self, X: np.ndarray) -> np.ndarray: ...

```

Required visualisation and analysis:

- For each dataset, compute the scree plot (cumulative explained variance).
- Project data to the first 2 PCs, then create scatter plots colored by (a) true class labels, (b) K-Means cluster labels, (c) DBSCAN cluster labels.

2.4.2 K-Means

```

1 class KMeans:
2     # n_clusters: int; max_iter=300; tol=1e-4; random_state: int|None
3     # .centroids_ : np.ndarray (n_clusters, n_features)
4     # .labels_ : np.ndarray (n_samples,)
5     # .inertia_ : float -- sum of squared distances to centroids

```



```

6 def __init__(self, n_clusters: int, max_iter: int = 300,
7               tol: float = 1e-4, random_state: int | None = None) -> None: ...
8 def fit(self, X: np.ndarray) -> "KMeans": ... # Lloyd's algorithm

```

Required visualisation and analysis:

- Elbow method: for $k = 1$ to 10 (with multiple restarts, e.g., 10, keeping the lowest inertia), plot inertia vs k .
- Report Adjusted Rand Index (ARI) against true labels using `sklearn.metrics.adjusted_rand_score`.

2.4.3 DBSCAN

```

1 class DBSCAN:
2     # eps: float -- neighbourhood radius; min_samples: int -- core-point threshold
3     # .labels_ : np.ndarray (n_samples,) -- cluster ids; -1 = noise
4     def __init__(self, eps: float, min_samples: int) -> None: ...
5     def fit(self, X: np.ndarray) -> "DBSCAN": ... # density-based clustering

```

Required visualisation and analysis:

- k-distance plot: for each point compute the distance to its k -th nearest neighbour (set $k = \text{min_samples}$). Sort distances ascending and plot; choose ε near the “knee”.
- Report ARI against true labels and the fraction of points labelled as noise.
- Compare cluster quality with K-Means.

The above interfaces are binding. All required visualisation steps (scree, elbow, k-distance) must be reproducible via `run_all.py`.

3. Experimental Study Design

3.1 Datasets

Use at least **three datasets** of varying characteristics. You must justify your selection in the report. Suggested sources with direct links are given below. **Important:** some dataset names appear in multiple repositories; use the exact links and note the version you downloaded.

Dataset	Task	Size	Link / Source
Breast Cancer Wisconsin	Binary	569	<code>sklearn.datasets</code>
Adult Income	Binary, imbalanced	48 842	UCI Repository
Covertypes (subset)	Multi-class	≥ 5000	UCI Repository
MNIST (2-class subset)	High-dim	≥ 5000	<code>sklearn.datasets</code> (<code>fetch_openml</code>)

You must include at least one dataset with severe class imbalance (minority class $\leq 1\%$ of total) and at least one high-dimensional dataset (>20 features).

Class Imbalance Treatment

If a dataset exhibits severe class imbalance, you are expected to apply and document an appropriate treatment strategy (e.g. random oversampling, SMOTE, class weights in the loss/impurity). This topic has been covered in a dedicated class; the report should briefly explain the chosen method and justify why it is suitable for the data.

3.2 Preprocessing

- Standardize numerical features to zero mean, unit variance (use `StandardScaler` from sklearn, or implement your own). Fit scaler on training data only, then transform test.
- Handle missing values by either removing rows or imputing (justify your choice).
- For multi-class datasets, you may binarize or keep as multi-class. AdaBoost must support binary classification; implement one-vs-rest or SAMME.R as needed.

3.3 Experiments to run

For each dataset, run and report all of the following. All random seeds must be fixed (e.g., 42) for reproducibility. Every experiment must be fully reproducible by `python src/experiments/run_all.py`.

1. Baseline.

- Train a single unpruned `DecisionTree` (your implementation) on an 80/20 train/test split.
- Train a depth-1 stump (`DecisionStump`).
- Train `sklearn.tree.DecisionTreeClassifier` with identical parameters.
- Report accuracy, F1 (macro), and AUC-ROC (for binary) or macro-averaged AUC for multi-class. Verify your tree matches sklearn within 2%.

2. AdaBoost scaling.

- Vary `n_estimators` from 1 to 200 (step 5 or 10).
- For each number, record test accuracy and training accuracy using `staged_predict`.
- Plot training error and test error vs. number of estimators. Observe if and when AdaBoost overfits.

3. Random Forest scaling.

- (a) Fix `max_depth=None`, vary `n_estimators` from 1 to 200. Plot test accuracy.
- (b) Fix `n_estimators=100`, vary `max_depth` from 1 to 20. Plot test accuracy.
- If `oob_score=True`, also plot OOB accuracy alongside test accuracy. Discuss the relationship.

4. Head-to-head comparison (fixed resources).

- Set `n_estimators = 100` for both ensembles.
- 5-fold cross-validation. For each fold compute accuracy, macro F1, and AUC-ROC.
- Report mean $\pm$ standard deviation for: single tree, AdaBoost, Random Forest, and `sklearn.ensemble.RandomForestClassifier` (as reference). Use box plots or tables.

5. Noise robustness.

- Randomly flip a fraction $\eta \in \{0.05, 0.10, 0.20\}$ of training labels.
- For each η , train AdaBoost (100 stumps) and Random Forest (100 trees) on corrupted data; evaluate on clean test set.
- Plot accuracy degradation curves. Discuss which is more sensitive and relate to bias-variance profiles.

6. Bias–variance decomposition.

- Select one balanced binary dataset. Generate $B = 100$ bootstrap replicates from training data.
- For each replicate train the model; evaluate on a large fixed test set.
- Compute bias² and variance as per definitions in Breiman (1996).
- Present as a table or bar chart. Explain how boosting reduces bias while bagging reduces variance.

7. Unsupervised analysis.

- Scree plot; choose PCs capturing $\geq 90\%$ variance.
- 2D PCA scatter plots colored by true class, K-Means clusters, and DBSCAN clusters.
- Report ARI for K-Means (best k) and DBSCAN (best ε). Include k-distance plot.
- Discuss alignment of clusters with class boundaries and correlation with classifier performance.

4. Project Format

4.1 Teams

Teams of 3 students (2 or 4 with prior written approval). Email group composition and dataset choices to sultan.musayeva@aiacademy.az by **Day 2** (June 23, 23:59 UTC+4).

4.2 Repository structure

```
team-<name>/
|-- README.md                # setup, dependencies, how to run
|-- requirements.txt         # pinned versions
|-- .gitignore
|-- src/
|   |-- __init__.py
|   |-- trees/
|   |   |-- decision_tree.py
|   |   |-- adaboost.py
|   |   |-- random_forest.py
|   |-- unsupervised/
|   |   |-- pca.py
|   |   |-- kmeans.py
|   |   |-- dbscan.py
|   |-- experiments/
|   |   |-- run_all.py
|   |   |-- utils.py
|-- tests/
|-- notebooks/
|-- figures/
|-- data/                    # .gitignore large files; include
|   download_data.sh
|-- report/
|   |-- report.tex
|   |-- report.pdf
|-- presentation/
|   |-- presentation.pdf     # slide deck (mandatory)
```

5. Deliverables

1. **GitHub repository** tagged `v1.0-final`. Must include source code, tests, `requirements.txt`, and `README.md` with one-command reproduction (`python src/experiments/run_all.py`).
2. **IEEE-style paper** (`report/report.pdf`) in two-column format, **6–8 pages** (excluding references and back matter). Every claim must be backed by a table/figure.
3. **Professional slide deck** (`presentation/presentation.pdf`). A 10–12 slide presentation (PDF) summarizing the project's motivation, methods, experiments, results, and conclusions. Must be submitted to Moodle prior to the defense.
4. **Paper defense**. Group oral presentation and examination. Each team gives a 10-minute presentation using their prepared slides, followed by a 10–12 minute Q&A session with the instructor. Individual understanding will be assessed during the Q&A.

Slide Deck Guidelines

Your slide deck is a key deliverable. It should be visually consistent, professional, and data-driven. Avoid text-heavy slides; use well-designed figures, tables, and concise bullet points. Submit the final PDF to Moodle before the defense. The slide deck will be graded according to the rubric.

6. Contribution Report

To ensure fair assessment of individual contributions, each team must submit a **contribution report** detailing the specific work performed by each member.

- The report should be a separate PDF document named `contribution_report.pdf` placed in the root of the repository and submitted to Moodle.
- It must include a table with columns: **Member Name**, **Modules/Code Sections**, **Report Sections**, **Experiments**, **Slides**, **Other Tasks**. Each member should provide a concise description of their contributions in each area.
- The report must be signed (or electronically agreed) by all team members to confirm its accuracy.

Verification. The instructor will cross-check the contribution report against GitHub commit statistics (lines added/removed, number of commits, and commit messages). Severe discrepancies between reported contributions and Git history will be investigated.

Penalty for Mismatch or Imbalance

If a team member's reported contribution is found to be significantly misrepresented, or if the division of work is grossly imbalanced (e.g., one member does > 70% of the work and the others barely contribute), the following penalties will apply:

- **5 points** for the entire team if the contribution report is missing or submitted late.
- **8 points** for a member who cannot substantiate their claimed contributions.
- In cases of severe imbalance, individual grades may be adjusted to reflect actual contributions; the total team grade may also be reduced.

The contribution report will be considered a mandatory part of the project deliverables. It will also be used during the defense to guide questions about individual understanding of each module.

7. Grading Rubric

Total: **100 points** (Code 45%, Report 25%, Presentation 10%, Defense 20%).

Criterion	Weight	Detailed expectations
Code – 45%		
Correctness of implementations	18%	Within 2% of sklearn baselines. All unit tests pass. No sklearn ensemble classes used.
Code quality & design	9%	OOP, type hints, docstrings, no magic numbers. <code>mypy</code> or <code>pyright</code> run.
Unsupervised pipeline	9%	PCA, K-Means, DBSCAN correct, required visualizations produced. ARI reported.
Testing	9%	<code>pytest</code> $\geq 60\%$ coverage; edge cases; deterministic with <code>random_state</code> .
Report – 25%		
Experimental rigor	10%	All 7 experiments present; cross-validation correct; bias-variance decomposition; noise experiment; tables/figures match code.
Conceptual depth	9%	Correct explanations of AdaBoost weight update, bootstrapping, Gini, cluster-classification relationship. Answers central question with evidence.
Clarity & format	6%	IEEE format, page limits, proper citations, no leftover template text.
Presentation – 10%		
Slide design	3%	Professional layout, consistent theme, readable fonts, no clutter.
Data visualization	3%	Clear, accurate figures. Captions and axes labeled.
Content organization	4%	Logical flow, covers all key aspects (motivation, methods, results, conclusion), meets time limit.
Defense – 20%		
Individual understanding	12%	Can explain and defend any part of the code. Can derive AdaBoost weight update.
Depth of analysis	5%	Reasons about failure modes, proposes improvements, explains tradeoffs.
Clarity under pressure	3%	Precise, concise, handles follow-up questions.

7.1 Automatic deductions

- Using `sklearn.tree.DecisionTreeClassifier` or `sklearn.ensemble.*` as your implementation: **–25 pts**.
- Team member unable to explain own module: **–8 pts per module**.
- Hard-coded file paths or magic constants: –5 pts.
- Test suite below 60% coverage: –5 pts.
- TODO/FIXME in final code: –3 pts.
- Missing `run_all.py` or non-reproducible figures: –5 pts.
- Report outside 6–8 page limit (excluding references): –3 pts.
- Missing slide deck or non-submission prior to defense: –5 pts.
- Missing contribution report or late submission: **–5 pts** (team deduction).
- Contribution report inconsistencies (as per §6): individual and/or team penalties.

8. Advanced Bonuses (Optional)

Up to **+10 bonus points**. Each bonus must be documented and tested.

Bonus	Points	What we look for
Gradient Boosting (GBM)	+4	Basic GBM with log-loss, comparison against AdaBoost.
t-SNE visualization	+2	Implement or use sklearn t-SNE; compare with PCA, explain differences.
Multiclass AdaBoost (SAMME.R)	+2	Real-valued variant, improved calibration on multi-class.
GitHub Actions CI	+1	Lint + type-check + test on every PR. Coverage gate at 60%.
Interactive figure notebook	+1	<code>notebooks/exploration.ipynb</code> with sliders and live plots.

9. What You May NOT Do

- Do not use `sklearn`’s `DecisionTreeClassifier`, `AdaBoostClassifier`, or `RandomForestClassifier` as implementations.
- Do not use `xgboost`, `lightgbm`, `catboost` for the main experiments.
- Do not share code between teams.
- AI-assisted coding must be disclosed; you must be able to defend every line.

10. Academic Integrity

Two rules for AI tools:

1. You must understand and be able to defend every line.
2. Disclose substantial AI assistance in the report’s “Tools & Acknowledgements” section.

Plagiarism detection will be run across all repositories.

11. Self-Assessment Checklist

Before tagging `v1.0-final`:

Implementations

- ☐ `DecisionTree` matches sklearn within 2%.
- ☐ `AdaBoostClassifier` weight update follows Freund & Schapire (1997).
- ☐ `RandomForestClassifier` OOB score computed.
- ☐ PCA, KMeans, DBSCAN all implemented without sklearn (except ARI).

Experiments

- ☐ All 7 experiments in §3 reproducible via `run_all.py`.
- ☐ Bias-variance decomposition present.
- ☐ Noise robustness includes 5%, 10%, 20% levels.
- ☐ K-distance plot; ε justified.

Code quality

- ☐ `pytest -cov` $\geq 60\%$.
- ☐ `mypy` or `pyright` run; result noted.
- ☐ No `TODO` or `FIXME`.
- ☐ All random operations seeded via `random_state`.

Report

- ☐ 6–8 pages, IEEE format.
- ☐ All required reading cited.
- ☐ Every figure has a caption; every table has a header.
- ☐ No leftover template text.

Presentation

- ☐ Slide deck (10-12 slides) submitted as `presentation.pdf`.
- ☐ Professional design and consistent theme.
- ☐ Figures, tables, and visualizations are clear and sourced from report code.

Contribution Report

- ☐ `contribution_report.pdf` placed in root directory and submitted to Moodle.
- ☐ All members have reviewed and signed the report.

Defense readiness

- ☐ Every member can derive AdaBoost weight update.
- ☐ Every member can walk through their module.
- ☐ Every member can interpret every figure.
- ☐ Every member has completed the required reading.

Implement it cleanly. Test it rigorously. Present it professionally. Understand it deeply.